

SYSTEM DESIGN INTERVIEW

Design Instagram

distributed photo-sharing platform at massive scale

~ 35 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Photo Upload Pipeline
- 06 · Database Design
- 07 · News Feed Architecture
- 08 · Feed Ranking & Personalization
- 09 · Stories Architecture
- 10 · Search & Explore
- 11 · Caching Strategy
- 12 · Notifications

- 13** · Scalability & Sharding
- 14** · Failure Modes & Recovery
- 15** · Design Trade-offs
- 16** · Interview Playbook

01

Problem Statement

Define the scope and constraints

Build a photo-sharing platform — **Instagram** — where users upload, share, and discover photos and videos in real-time at billion-user scale. The design must handle uploads, personalised feeds, Stories, search, and notifications without breaking under fan-out load.

Key Features

- Users upload photos/videos and compose captions with hashtags + location
- Users follow others; see a personalised feed ranked by relevance
- Like, comment, share — engagement primitives drive ranking
- Stories — ephemeral 24-hour content with automatic TTL
- Search by hashtags, usernames, captions
- Explore — trending content surfaced algorithmically
- Notifications — likes, comments, follows, mentions

Clarifying Questions

Question	Answer
Scope?	Photos + videos + Stories; DMs out of scope for MVP
Scale?	500M DAU, 100M photos/day
Geography?	Global; multi-region active-active reads
Latency?	Feed load < 2s; upload acknowledged < 5s
Resolution?	Original + 3 thumbnail variants (320, 640, 1080)
Retention?	Photos kept indefinitely; Stories auto-expire at 24h
Real-time?	Notifications < 1s; feed eventually consistent

KEY

Key Assumption

We design for **500M DAU**, **100M photos/day**, global scale, < 2s feed latency, real-time notifications. Everything else follows from those four numbers.

02

Requirements

Functional and non-functional

Functional Requirements

- **Upload:** photos/videos with caption, location, hashtags
- **Feed:** personalised home feed based on follows + ranking
- **Engagement:** like, unlike, comment, share
- **Search:** by hashtag, username, caption text
- **Stories:** ephemeral 24h content with view receipts
- **Social graph:** follow / unfollow; mutual-follow reciprocity
- **Notifications:** push for likes, comments, follows, mentions

Non-Functional Requirements

Requirement	Target
Availability	99.9% uptime ($\approx$ 4.3 min/month downtime)
Latency	Feed < 2s p99; photo upload ack < 5s
Durability	11 nines on photos (S3); zero loss of user data
Consistency	Eventual for feed; strong for user/auth/payment
Throughput	100M photos/day; ~29K feed reads/sec sustained
Scalability	Handle 10 $\times$ growth without redesign
Cost	Optimise for storage + egress (the dominant line items)
Security	Auth, encryption-in-transit, rate limiting, abuse prevention

INSIGHT

Availability vs. Consistency

Feed service is eventually consistent — a stale read that misses the very latest post for a few seconds is fine. Account, auth, and payment data are strongly consistent: you cannot show a user the wrong balance or the wrong phone number even briefly.

03

Capacity Estimation

Math for scale

Traffic Estimates

- DAU: **500M**
- Uploads: **100M photos/day**
- Average feeds viewed per user per day: **5**
- Feed read QPS: $500M \times 5 / 86,400 \text{ sec} = \mathbf{28,935 \approx 29K \text{ reads/sec}}$
- Upload QPS: $100M / 86,400 = \mathbf{1,157 \approx 1.16K \text{ uploads/sec}}$
- Like/comment QPS: $\sim 10\times \text{upload rate} \approx \mathbf{12K \text{ writes/sec}}$ (engagement)

Storage Estimates

- Average photo size (compressed original): **200 KB**
- Daily raw uploads: $100M \times 200 \text{ KB} = \mathbf{20 \text{ TB/day}}$
- Variants (3 thumbnails @ 30/60/120 KB $\approx 210 \text{ KB}$ extra): **$\sim 3\times$ multiplier $\rightarrow \sim 60 \text{ TB/day all-in}$**
- Annual storage (with variants): $60 \text{ TB} \times 365 = \mathbf{\sim 22 \text{ PB/year}}$
- 5-year corpus: **$\sim 110 \text{ PB}$** on S3 (Standard + Glacier tiering)

Bandwidth Estimates (recomputed)

- Average rendered photo size on the wire (resized + compressed): **100 KB**
- Daily feed photo loads: $500M \text{ DAU} \times 5 \text{ feeds} \times \sim 10 \text{ photos} = \mathbf{25B \text{ images/day}}$
- Outbound egress: $25B \times 100 \text{ KB} = \mathbf{2.5 \text{ PB/day} \approx 231 \text{ Gbps average}}$
- Inbound (uploads): $100M \times 200 \text{ KB} = 20 \text{ TB/day} = \mathbf{\sim 1.85 \text{ Gbps average}}$
- Peak (3 $\times$ diurnal skew): **$\sim 700 \text{ Gbps egress}$, $\sim 5.5 \text{ Gbps ingress}$**
- **CDN absorbs $\geq 95\%$ of egress at the edge $\rightarrow$ origin pull $\approx 12 \text{ Gbps average}$**

WARNING

Bandwidth — Watch Your Units

An earlier rev of this guide quoted 2.3 Gbps egress. That number divided bytes by seconds and forgot the $\times 8$ to get bits, so it was off by $\sim 10\times$. Real number: $250 \text{ TB/day} \times 8 / 86,400 \approx \mathbf{23 \text{ Gbps}}$ at 10 photos/feed $\times 100 \text{ KB}$; at 25B images/day it's **$\sim 231 \text{ Gbps}$** . Always sanity-check Gbps vs. GB/s — they differ by a factor of 8.

Storage Strategy

TIP

Where Each Byte Lives

Originals → **S3** (durable, cheap-ish at scale). Thumbnails → **CDN edges** (fast serving). User/post metadata → **MySQL** (sharded, strong consistency). Feed timelines + follows → **Cassandra** (wide rows, eventual). Hot reads → **Redis**. Search → **Elasticsearch**. Async work → **Kafka**.

KEY

Key Numbers to Memorise

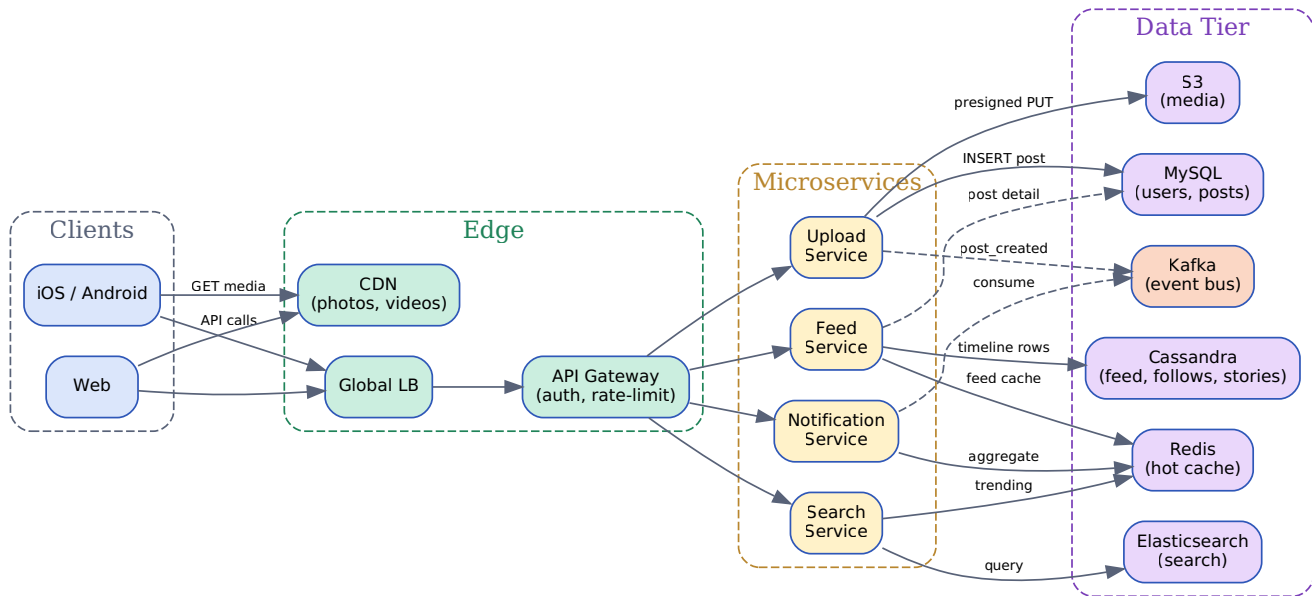
500M DAU · **100M** photos/day (1.16K/sec) · **29K** feed reads/sec · **20 TB/day** raw, **60 TB/day** with variants · **~231 Gbps** egress average, **~700 Gbps** peak · **110 PB** over 5 years.

04

High-Level Architecture

System overview

Instagram is a layered service-oriented architecture. Clients hit a global edge (CDN + load balancer), authenticate at an API gateway, then talk to independent microservices — Upload, Feed, Search, Notifications. Each service owns its primary store and communicates with siblings via Kafka events. No service shares a database with another.



Top-level flow: clients hit the CDN/LB, auth at the gateway, microservices fan out to purpose-built data stores. Kafka decouples services for async work.

Layer Responsibilities

- **Client SDK:** retries, image compression before upload, prefetches the next 20 feed posts
- **CDN:** serves all photo/video bytes; absorbs > 95% of read egress
- **API Gateway:** JWT auth, per-user/IP rate limits, request routing, request shaping
- **Upload Service:** issues presigned S3 URLs; never sees the bytes itself
- **Feed Service:** hybrid fan-out; reads pre-computed timelines from Cassandra/Redis
- **Search Service:** Elasticsearch for full-text + tag queries; Redis for trending
- **Notification Service:** consumes Kafka events; aggregates; pushes via APNs/FCM

INSIGHT

Design Philosophy

Every service owns its own data store and communicates only via async events on Kafka. No shared databases, no synchronous cross-service calls in the write path. This is what lets Feed, Search, and Notifications scale independently and survive each other's outages.

05

Photo Upload Pipeline

Presigned URLs and async processing

Uploading a 5–20 MB photo through your service tier is wasteful — the bytes ride the request twice (client → API server → S3). The fix: **presigned S3 URLs**. Server hands the client a time-limited URL; client PUTs directly to S3; an S3 event triggers async processing.

Step-by-Step Flow

1. Client calls POST /upload with metadata (caption, location, hashtags)
2. Upload Service returns a **presigned S3 URL**, valid for 15 minutes
3. Client PUTs the photo bytes directly to S3 — service tier never sees them
4. S3 fires an event: S3 → SNS → SQS → Kafka topic media_uploaded
5. Media Processor consumes the event; downloads the original from S3
6. Generates 3 thumbnails (320×320, 640×640, 1080×1080); stores variants in S3
7. CDN is pre-warmed by issuing a HEAD on each variant from edge POPs
8. DB row marked status = ready; post_created emitted to Kafka
9. Fan-Out Worker picks up the event; pushes into followers' feeds (see §07)

TIP

Why Presigned URLs?

Offloads upload bandwidth from the service tier to S3 directly. At 1.16K uploads/sec × 200 KB = 1.85 Gbps inbound, the API tier would burn an entire service shard just shovelling bytes. Presigned URLs cut that to zero. Bonus: S3 supports resumable multipart uploads — flaky mobile networks just retry.

Media Processor — What It Actually Does

- **Validate:** EXIF strip, MIME sniff, dimension limits, NSFW classifier
- **Transcode:** re-encode to optimised JPEG (mozjpeg) / HEIC / WebP
- **Thumbnails:** 320 (grid), 640 (feed), 1080 (full-screen)
- **Video:** ffmpeg → H.264 / H.265 with adaptive bitrate ladders
- **Perceptual hash:** pHash for dedup + abuse detection
- **CDN warm:** push to edges so first follower view is < 50 ms

```
# Issue a presigned PUT URL (server side, boto3)
url = s3.generate_presigned_url(
    'put_object',
    Params={
```

```
        'Bucket':      'ig-uploads',
        'Key':         f'raw/{user_id}/{post_id}.jpg',
        'ContentType': 'image/jpeg',
    },
    ExpiresIn=900, # 15 minutes
)
return {'upload_url': url, 'post_id': post_id}
```

06

Database Design

Polyglot persistence

No single store fits every access pattern. We pick one per pattern: **MySQL** for users and posts (strong consistency, transactional writes), **Cassandra** for follows and feed timelines (write-heavy, wide rows, multi-region), **Redis** for hot reads, **S3** for media bytes.

users (MySQL — sharded by user_id)

```
CREATE TABLE users (  
  user_id          BIGINT PRIMARY KEY,  
  username         VARCHAR(255) UNIQUE NOT NULL,  
  email            VARCHAR(255) UNIQUE NOT NULL,  
  hashed_password  VARCHAR(255),  
  avatar_url       VARCHAR(2048),  
  bio              TEXT,  
  follower_count   INT DEFAULT 0,  
  following_count  INT DEFAULT 0,  
  created_at       TIMESTAMP,  
  updated_at       TIMESTAMP  
);  
-- Sharded: hash(user_id) % 256
```

posts (MySQL — sharded by user_id)

```
CREATE TABLE posts (  
  post_id          BIGINT PRIMARY KEY,  
  user_id          BIGINT NOT NULL,  
  caption          TEXT,  
  photo_urls       JSON,                -- list of S3 keys  
  location         VARCHAR(255),  
  hashtags         JSON,                -- list of strings  
  like_count       INT DEFAULT 0,  
  comment_count    INT DEFAULT 0,  
  created_at       TIMESTAMP,  
  INDEX idx_user_created (user_id, created_at DESC)  
);  
-- Co-locate a user's posts on the same shard for profile reads
```

follows + reverse_follows (Cassandra)

```
CREATE TABLE follows (  
  follower_id  BIGINT,  
  following_id  BIGINT,  
  created_at    TIMESTAMP,  
  PRIMARY KEY (follower_id, following_id)  
);  
  
CREATE TABLE reverse_follows (  
  following_id  BIGINT,  
  follower_id  BIGINT,  
  created_at    TIMESTAMP,  
  PRIMARY KEY (following_id, follower_id)  
);  
-- Two tables: 'who do I follow?' and 'who follows me?' both O(1).
```

feed_timeline (Cassandra — wide row per user)

```
CREATE TABLE feed_timeline (  
  user_id      BIGINT,  
  created_at    TIMESTAMP,  
  post_id      BIGINT,  
  post_user_id BIGINT,  
  PRIMARY KEY (user_id, created_at, post_id)  
) WITH CLUSTERING ORDER BY (created_at DESC);  
-- Denormalised: one row per (user, post) on each follower's timeline
```

INSIGHT

Denormalisation Trade-off

Duplicating posts across all followers' feeds is **write amplification**: a single post by a user with 1,000 followers becomes 1,000 Cassandra inserts. We pay that write cost so the read path is a single partition lookup. At 100:1 read:write ratios this is the right trade — but it's the reason \$07 exists.

07

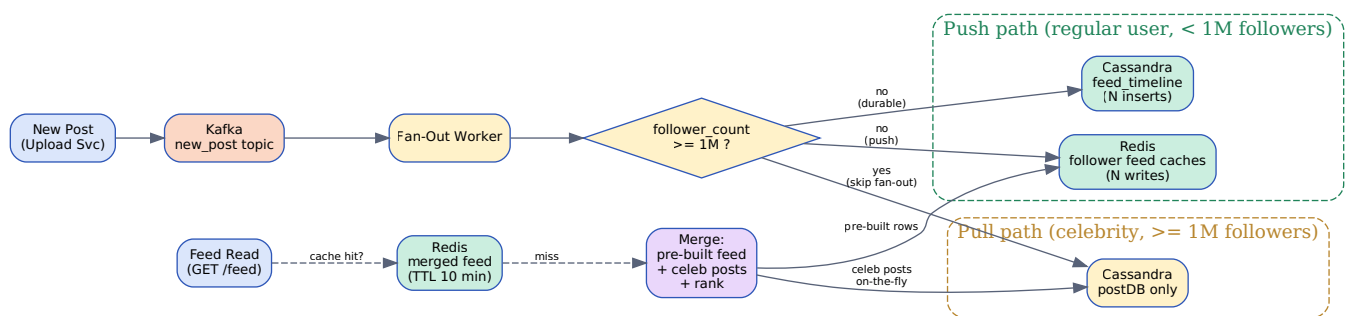
News Feed Architecture

Hybrid push / pull fan-out

Building a real-time feed for 500M users is the hardest problem in this design. Two pure approaches exist — **fan-out on write (push)** and **fan-out on read (pull)**. Neither works alone at the Kardashian end of the follower distribution. Real systems use a hybrid.

Push vs. Pull vs. Hybrid

Aspect	Push (fan-out on write)	Pull (fan-out on read)	Hybrid (recommended)
Read latency	Fast — feed pre-built	Slow — merge per request	Fast for both paths
Write cost	$O(\text{followers})$ per post	$O(1)$ per post	$O(\text{followers})$ for $< 1\text{M}$; $O(1)$ for $\geq 1\text{M}$
Storage	N copies (one per follower)	1 copy (in postDB)	Mostly push, celebrities pull
Worst case	Celebrity post $\rightarrow$ 100M writes	Active user opens app $\rightarrow$ merge 1000s	Celebrity post: $O(1)$; user open: merge ~ 50 celebs
Consistency	Eventually consistent across followers	Always fresh on read	Push lag $< 5\text{s}$; pull always fresh



Hybrid fan-out: regular users push into followers' Redis caches; celebrities skip the push and live in Cassandra postDB; the read path merges both at request time.

Hybrid Fan-Out Algorithm

1. User A posts $\rightarrow$ Upload Service emits `new_post` on Kafka
2. Fan-Out Worker consumes the event; reads A's `follower_count` from MySQL

3. If **follower_count** < **1M**: push (post_id, A, ts) into each follower's Redis feed list and Cassandra feed_timeline
4. If **follower_count** ≥ **1M**: skip push entirely; the post is already in Cassandra postDB
5. On feed read: hit Redis for the pre-built rows from non-celeb follows; in parallel, query postDB for the celebs the user follows; merge by score; cache the merged result in Redis for 10 minutes

KEY

Write-Amp vs. Merge-on-Read — the 1M Threshold

Why is 1M the magic number? Run the math.

Push cost for one celeb post (1M followers): 1,000,000 Redis SETs + 1,000,000 Cassandra inserts. At typical 50µs Redis SET + 200µs Cassandra write, that's **~250 sec of cluster work** for one post. With 100M posts/day total and assume 0.01% are celebs (10K celeb posts/day) at 1M followers each → **10B push ops/day** just for celebrities, saturating the Redis cluster.

Pull cost (skip push, merge on read): 500M DAU × 5 feed loads × ~50 celebs followed avg = 125B partition reads/day on Cassandra postDB. Each is a single-key wide-row read, ~1 ms with Redis cache in front (≥ 90% hit rate after first read). Net: ~12.5B disk reads/day, well within a Cassandra cluster's budget.

Verdict: push wins by 10× for users below 1M followers; pull wins by 100× above it. The 1M threshold is where the curves cross.

INSIGHT

Why Hybrid?

1M is a breakpoint, not a magic number — it tracks the cost-cross-over above. Regular users (≈ 99.99% of accounts) get push-based instant feeds. The ~50K celebrities get pull. Result: 99.9% of reads are a single Redis lookup; the rest pay one extra Cassandra round-trip and merge.

08

Feed Ranking & Personalization

EdgeRank-style scoring

Pure chronological feeds bury the good stuff. Instagram's ranker scores posts on engagement, recency, relationship strength, and content type, then re-orders the merged candidate set. Models train offline daily; scoring at serve time is a fast linear combination.

Ranking Signals

Signal	Effect
Engagement	Likes + 2× comments + 3× shares; normalised by follower count
Recency	Exponential decay — newer wins; half-life ≈ 2 hours
Relationship strength	How often this user interacts with the poster
Content type	Video > photo > text; Stories scored separately
User preference	ML learns hashtags / accounts / locations the user engages with
Time zone	Boost posts from accounts active in overlapping windows

EdgeRank-style Formula

```
score = (  
    (1.0 + engagement_factor)  
    * recency_decay(hours_since_post)  
    * relationship_strength  
    * content_type_boost  
)  
  
engagement_factor    = (likes + 2*comments + 3*shares) / max(100, follower_count)  
recency_decay(h)     = 1.0 / (1 + 0.5 * h)  
relationship_strength = log(1 + interaction_count) / 10  
content_type_boost   = {'video': 1.3, 'photo': 1.0, 'carousel': 1.15}[type]  
  
# Real production has 100+ signals fed to a GBDT or two-tower model
```

TIP

Offline Training, Online Scoring

Models train on the previous day's engagement logs in a Spark/PyTorch pipeline. The serving tier loads frozen weights and computes a score in < 1 ms per candidate. Don't try to train online during feed reads — the latency budget is too tight.

09

Stories Architecture

Ephemeral content with TTL

Stories are 24-hour photos/videos that auto-delete. They drive higher engagement than feed posts — ephemeral FOMO works. The architecture is deliberately distinct from the feed: different access pattern, different lifecycle, no ranking.

Design Decisions

- **Separate Cassandra table** with native TTL — rows tombstone after 86,400 sec
- **Partition by user_id, cluster by created_at DESC** — fast retrieval of a user's recent stories
- **Denormalise** the poster's avatar + name into the story row to avoid join on read
- **View receipts** tracked in a counter table; visible only to the poster
- **Strict chronological order** — no ranker; users expect linear time
- **CDN TTL = 24h** — auto-evicts as Cassandra TTL expires

Cassandra Schema

```
CREATE TABLE stories (  
  user_id      BIGINT,  
  created_at   TIMESTAMP,  
  story_id     BIGINT,  
  media_url    VARCHAR(2048),  
  caption      TEXT,  
  poster_name  VARCHAR(255),    -- denormalised  
  poster_avatar VARCHAR(2048),  -- denormalised  
  PRIMARY KEY (user_id, created_at, story_id)  
) WITH CLUSTERING ORDER BY (created_at DESC)  
   AND default_time_to_live = 86400; -- 24h auto-delete
```

View Tracking

```
-- Detailed who-viewed-what (visible to poster only)  
CREATE TABLE story_views (  
  story_id  BIGINT,  
  viewed_at TIMESTAMP,  
  viewer_id BIGINT,  
  PRIMARY KEY (story_id, viewed_at, viewer_id)  
) WITH default_time_to_live = 86400;  
  
-- Fast view-count aggregate via Cassandra counter
```

```
CREATE TABLE story_view_counts (  
  story_id BIGINT PRIMARY KEY,  
  view_count COUNTER  
);
```

INSIGHT

Why a Separate Table?

Stories have *fundamentally* different access patterns: short TTL, no engagement ranking, strict order, view receipts. Co-locating with feed posts would force one schema to optimise for both — and lose on each. Separate tables let each side pick the right partition key, TTL, and compaction strategy.

10

Search & Explore

Elasticsearch + trending

Search hits Elasticsearch — full-text on captions, keyword on hashtags and usernames, range filters on date and engagement. Explore ranks trending hashtags via Redis sorted sets, refreshed hourly per region.

Posts Index Mapping

```
PUT /posts
{
  "mappings": {
    "properties": {
      "post_id": { "type": "keyword" },
      "user_id": { "type": "keyword" },
      "caption": { "type": "text", "analyzer": "standard" },
      "hashtags": { "type": "keyword" },
      "location": { "type": "geo_point" },
      "created_at": { "type": "date" },
      "like_count": { "type": "integer" }
    }
  }
}
```

Sample Query

```
GET /posts/_search
{
  "query": {
    "bool": {
      "must": [{ "match": { "caption": "pizza" } }],
      "filter": [
        { "terms": { "hashtags": ["#food", "#nyc"] } },
        { "range": { "created_at": { "gte": "now-30d" } } }
      ]
    }
  },
  "sort": [{ "like_count": "desc" }],
  "size": 20
}

# 'pizza' posts tagged #food or #nyc in the last 30 days
```

Explore — Trending Hashtags

- **Redis sorted set** per region; score = view + like count
- **Hourly windows** — separate keys per hour, easier rollups
- **Decay**: yesterday's scores weighted $\frac{1}{2}$, day-before $\frac{1}{4}$ — exponential
- **Personalisation**: ZUNIONSTORE trending n user interest vector

```
# Increment trending score on each tag use
ZINCRBY trending:global:2026-05-06-10 1 "#pizza"
ZINCRBY trending:us-ny:2026-05-06-10 1 "#nyc"

# Top 20 trending globally right now
ZREVRANGE trending:global:2026-05-06-10 0 19 WITHSCORES

# Personalised: combine global trends with user interest vector
ZUNIONSTORE explore:user:123 2 \
    trending:global:2026-05-06-10 user:123:interests \
    WEIGHTS 0.6 0.4
```

11

Caching Strategy

What, where, and TTL

Caching is what makes 29K reads/sec affordable. We cache at four layers: client SDK (prefetch the next 20 posts), CDN (photo bytes), Redis (metadata, feed lists, trending), and the database query cache. Each has a different TTL tuned to staleness tolerance.

Cache Layers

Layer	What	Where	TTL	Reasoning
CDN	Photo / video bytes	Cloudflare / Akamai edge	7 days	Reduce origin S3 egress by ~95%
Redis	User profile (name, bio, counts)	Redis cluster sharded by user_id	1 hour	Profile updates rare; 1h staleness fine
Redis	Feed timeline (list of post_ids)	Redis cluster	10 min	Feeds change fast; refresh often
Redis	Post metadata (caption, like, comment counts)	Redis cluster	30 min	Engagement counts change frequently
Redis	Trending hashtags	Redis sorted sets per region	1 hour	Trends shift hourly
Client SDK	Pre-fetched next 20 feed items	Mobile device	Session	Reduce API calls during scroll

INSIGHT

Cache Invalidation Strategies

TTL-based (simple, eventual): refresh every N minutes — tolerable for everything we cache. **Event-based** (accurate): on post update, publish to Kafka; cache service deletes the affected keys. **Hybrid** (both): event-based for accuracy, TTL as a backstop if the event is dropped. We use hybrid for posts and profiles.

Event-based Invalidation

```
# Post-update path emits invalidation event
POST /posts/{post_id}
  1. Update MySQL row
  2. Publish post_updated event to Kafka
```

```
# Cache invalidator consumer
for event in kafka_consumer:
    pid = event['post_id']
    redis.delete(f'post:meta:{pid}')
    redis.delete(f'post:comments:{pid}')
    # Affected feed caches expire on TTL – too many to enumerate
```

12

Notifications

Likes, comments, follows, mentions

Real-time notifications drive engagement. When User A likes User B's post, B's phone should buzz within a second. The hard part isn't delivery — it's **aggregation and rate limiting** so users aren't spammed.

Notification Flow

1. User A likes B's post → Like Service emits `post_liked` on Kafka
2. Notification Service consumes; reads post → resolves author = B
3. Aggregator checks Redis: have we notified B about this post in the last hour?
4. Aggregate: *"Alice and 7 others liked your post"* instead of 8 separate pings
5. Dedupe: skip if B was already notified within 5 min
6. Hand off to APNs (iOS) / FCM (Android) push providers
7. User taps banner → app opens directly to the post

Aggregation Logic

```
class NotificationService:
    def on_like(self, ev: LikeEvent):
        post = db.get_post(ev.post_id)
        author = post.user_id

        # pull existing aggregate window
        key = f'notif:{author}:post:{ev.post_id}'
        existing = redis.get(key)

        if existing and time.time() - existing.last_sent < 300:
            # within 5 min – append actor, defer
            existing.actors.append(ev.user_id)
            redis.set(key, existing, ex=3600)
            return

        agg = self.aggregate(ev, existing)
        self.send_to_apns(author, agg)
        redis.set(key, agg, ex=3600)
```

WARNING

Filtering & Rate Limiting

Per-user mute lists in Redis prevent unwanted senders. Hard cap of **10 push notifications/min/user** — past that, batch. Notification fatigue is the leading cause of users disabling pushes entirely; protect that channel.

13

Scalability & Sharding

Horizontal scaling strategy

At 500M DAU, no single database fits. We shard everything horizontally and co-locate related rows on the same shard so reads stay single-shard.

Sharding Plan

Table	Shard Key	Shards	Why
users	hash(user_id) % 256	256+	Even distribution; scale users independently
posts	hash(user_id) % 256	256+	Co-locate user's posts on same shard for profile view
follows	hash(follower_id) % 256	256+	Fast 'who do I follow?' lookup; reverse_follows mirrors
feed_timeline (Cass)	hash(user_id) % 1024	1024+	Hottest table — needs more shards for fan-out throughput
comments	hash(post_id) % 512	512+	Group all comments per post together

WARNING

Resharding Pain

If we go from 256 → 512 shards via plain modulo, ~50% of keys move. Use **consistent hashing** (Ketama / jump hash) so only 1/N keys migrate when a node is added — a 2× scale-out moves ~50% with modulo but only ~6% with consistent hashing if doubling ring size.

Replication

- **MySQL:** 1 primary + 3 replicas per shard; semi-sync replication; ~100 ms lag
- **Cassandra:** RF = 3 across AZs; LOCAL_QUORUM reads/writes
- **Redis:** primary-replica pairs; AOF persistence; cross-AZ failover
- **S3:** cross-region replication for the originals bucket

Connection Pooling

```
# Pool per shard — never open a fresh TCP for each query
pool = MySQLConnectionPool(
```

```
pool_size=50,  
max_retries=3,  
retry_delay=0.1,  
)  
  
shard_id = hash(user_id) % 256  
with pool.get(shard_id) as conn:  
    return conn.execute(query)
```

INSIGHT

Global Distribution

Primary writes pinned to one region per user (locality of access). Reads served from the nearest regional replica. Cross-region replication via Cassandra and S3 cross-region; lag < 1 sec typical.

14

Failure Modes & Recovery

Graceful degradation

Distributed systems fail constantly. The design assumption is not that components stay up — it's that any single failure degrades a feature, not the whole site.

Failure	Impact	Mitigation
MySQL shard down	Reads/writes to that shard fail	Auto-promote replica (Orchestrator); reroute hot keys; ~30s RTO
Cassandra node down	Up to 1/3 of feed reads slower	RF=3 + LOCAL_QUORUM tolerates 1 down; gossip detects within seconds
Redis cluster down	5–10× MySQL/Cassandra load	Failover to replica; warm standby; degrade to direct DB read
CDN edge down	+50 ms latency in that region	DNS failover to backup CDN; origin S3 absorbs the spike
API Gateway failure	One instance loses traffic	Auto-scaling group; LB drops unhealthy node within 30s
Kafka broker down	Async pipeline lags	RF=3 brokers; in-flight messages buffered up to 7 days
Elasticsearch down	Search unavailable	Core features unaffected; show cached trending; ~30 min RTO
Network partition	Cross-region writes blocked	Cassandra QUORUM-on-AZ; consistency over availability for the money path
Cascading failure	Slow downstream collapses upstream	Circuit breaker — return cached feed when Feed Service slow
Data corruption	Incorrect data persisted	Versioned backups every 6h; PITR within 7 days

TIP

Circuit Breaker

When a downstream service violates SLA (e.g. Feed Service p99 > 2s), the API Gateway opens a circuit breaker and returns cached feed instead of waiting. The damaged service gets time to recover; users see slightly stale data instead of a spinner. Half-open probes test recovery.

WARNING

Graceful Degradation Order

If we have to drop features to keep the site up, the order is: 1) ranking falls back to chronological; 2) trending falls back to yesterday's snapshot; 3) notifications buffer to Kafka and drain late; 4) search returns 'temporarily unavailable'; 5) feed reads + photo views are **never** dropped.

15

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Feed consistency	Eventual	Stale reads briefly; fast reads. Strong consistency would slow the read path 5–10×.
Fan-out model	Hybrid push/pull	Push for < 1M followers (fast reads), pull for celebrities (cheap writes). Pure push fails on celebrities; pure pull fails on regular users at read time.
Storage layout	Denormalised feed	Write amp (N copies) but O(1) reads. Normalised would force expensive joins per feed read.
Cache TTL	1h profiles, 10min feeds	Sweet spot between freshness and DB load. Short TTL → 5–10× DB pressure; long TTL → stale UX.
Architecture	Microservices	Independent scaling, fault isolation. Operational overhead vs. monolith — at this scale, services win.
DB choice	MySQL + Cassandra	MySQL for transactional rows, Cassandra for write-heavy wide rows. One store cannot do both well.
Upload path	Presigned S3 URLs	Bytes never touch the service tier. Saves ~1.85 Gbps ingress, ~\$1M/year in NAT egress.
Stories storage	Separate Cassandra table with TTL	Different access pattern from feed; co-locating would compromise both schemas.

INSIGHT

No Silver Bullet

Every choice is a trade-off. The interview signal isn't whether you pick the 'right' answer — it's whether you can articulate the trade and defend it. Know your constraints (scale, consistency, latency, cost), then argue from those.

16

Interview Playbook

45-minute execution

45 minutes is tight. You must cover requirements, capacity, architecture, the feed deep-dive, scalability, and trade-offs — without rushing any. This playbook gives you a tested timing.

Time Budget

Window	Phase	Goal
00:00–02:00	Clarify requirements	Pin down scope: photos vs. videos, DAU, real-time?
02:00–07:00	Capacity estimation	500M DAU → 29K QPS, 100M photos → 20 TB/day, 231 Gbps egress
07:00–20:00	High-level architecture	Boxes: client → CDN → LB → GW → services → data tier
20:00–35:00	Deep dive on feed	Push vs. pull, 1M threshold, hybrid math
35:00–40:00	Remaining components	Upload, search, notifications — quick passes
40:00–45:00	Trade-offs + recovery	Failure modes, sharding, consistency choices

Key Talking Points

- **“500M DAU × 5 feeds / 86,400 = 29K QPS”** — capacity math, recited
- **“100M photos × 200 KB = 20 TB/day raw, ~110 PB over 5 years”**
- **“Presigned URLs offload upload bandwidth from the service tier”**
- **“Hybrid fan-out — push under 1M, pull over”**
- **“Cassandra for fan-out wide rows, MySQL for transactional posts”**
- **“Stories: separate table with native TTL = 86,400”**
- **“CDN absorbs ≥ 95% of egress; origin pull ~12 Gbps”**
- **“Eventual consistency for feed; strong for auth and money”**

Common Follow-ups

- **Q: What if a regular user crosses 1M followers?** A: A threshold service flips them to pull mode; their existing pre-built timelines stay valid for the next 24h, then the cache expires naturally.
- **Q: How do you handle a celebrity's celebrity?** A: Same — pull. Their post-DB partition is read on demand. Redis caches the merged feed for 10 min.
- **Q: How do you scale beyond 256 shards?** A: Consistent hashing minimises migration; resharding tool runs online; reads dual-write during cutover.
- **Q: What if Cassandra loses a partition?** A: RF=3 + QUORUM survives one node; gossip detects within seconds; hinted handoff catches up.
- **Q: Why not exactly-once notifications?** A: Cross-system exactly-once is unimplementable; we deliver at-least-once and dedupe at the receiver via event_id.

KEY

Numbers to Memorise

500M DAU · **100M photos/day** (1.16K/sec) · **29K** feed reads/sec · **20 TB/day** raw, **60 TB/day** with variants · **~231 Gbps** egress, **~1.85 Gbps** ingress · **110 PB** over 5 years · **1M-follower** threshold for push/pull · **86,400 s** Stories TTL.

TIP

If You Get Stuck

Don't panic. Restate the assumption you're working under. "If celebs are 0.01% of users, push costs X; otherwise Y." Articulating the branch-point shows the interviewer you're thinking about the cost surface, not memorising an answer.